WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Logic Gates

Lecture 06: NOT, AND, OR, NAND, NOR, XOR, XNOR, and universal gate sets

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Logic gates

02 From expressions to circuits

03 Universal gate sets

04 Summary

01

Logic gates

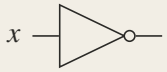
What is a logic gate?

> **Definition (Logic gate)**
>
> A *logic gate* is a piece of hardware that computes one of the basic Boolean functions: addition, multiplication, negation, or a simple combination of these.

- Gates are connected by wires into circuits, which compute larger functions.
- In a real chip, 0 and 1 are a low and a high voltage, and each gate is a handful of transistors: the switches of Lecture 01.
- From now on we work one layer up: with gates, not with transistors.

`Every gate in this lecture is in the Gates library of Logisim-evolution.`

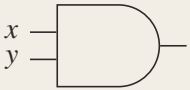
NOT, AND and OR



NOT

x'

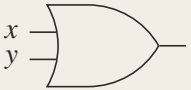
x	
0	1
1	0



AND

$x \cdot y$

x	y	
0	0	0
0	1	0
1	0	0
1	1	1



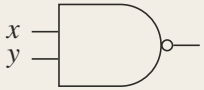
OR

$x + y$

x	y	
0	0	0
0	1	1
1	0	1
1	1	1

- The three operations of Boolean algebra (Lecture 03), now as hardware.
- The small circle on the NOT gate, the *bubble*, means complement. Without it, the triangle is a *buffer*: its output equals its input.

NAND and NOR



NAND

$(xy)' = x' + y'$

x	y	
0	0	1
0	1	1
1	0	1
1	1	0



NOR

$(x + y)' = x'y'$

x	y	
0	0	1
0	1	0
1	0	0
1	1	0

- NAND is NOT-AND: an AND gate with a bubble on its output. NOR is NOT-OR.
- The second form of each expression is De Morgan's theorem (Lecture 03).
- Each one, on its own, can build every circuit: see the last section.

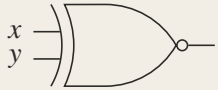
XOR and XNOR



XOR

$x \oplus y = x'y + xy'$

x	y	
0	0	0
0	1	1
1	0	1
1	1	0



XNOR

$(x \oplus y)' = x'y' + xy$

x	y	
0	0	1
0	1	0
1	0	0
1	1	1

- XOR, *exclusive OR*: ‘either x or y, but not both’. It is 1 when the inputs differ.
- XNOR is NOT-XOR: 1 when the inputs are equal.
- With more inputs, XOR is 1 when an odd number of them are 1.

All sixteen functions of two variables

xy = 00, 01, 10, 11					xy = 00, 01, 10, 11				
0	0	0	0	0	1	0	0	0	$(x+y)'$ NOR
0	0	0	1	xy AND	1	0	0	1	$x'y'+xy$ XNOR
0	0	1	0	xy'	1	0	1	0	y'
0	0	1	1	x	1	0	1	1	$x+y'$
0	1	0	0	$x'y$	1	1	0	0	x'
0	1	0	1	y	1	1	0	1	$x'+y$
0	1	1	0	$x'y+xy'$ XOR	1	1	1	0	$(xy)'$ NAND
0	1	1	1	$x+y$ OR	1	1	1	1	1

These are the $2^{2^2} = 16$ functions counted in Lecture 04. Six of them are the two-input gates of this lecture, in complementary pairs: AND and NAND, OR and NOR, XOR and XNOR.

Gates with more inputs

- An n-input AND is 1 when *all* its inputs are 1; an n-input OR when *at least one* is.
- Because AND and OR are associative, a 3-input AND is the same as two 2-input ANDs in a row: $xyz = (xy)z$.
- A 3-input NAND is $(xyz)'$, a NAND with three inputs.

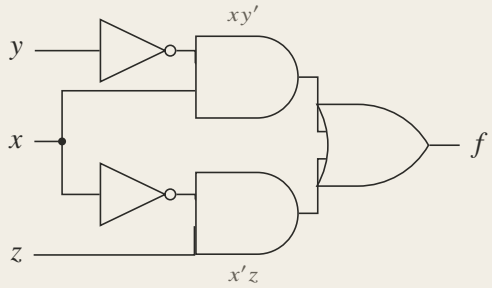
> **NAND and NOR are not associative**
>
> Two 2-input NANDs in a row are not a 3-input NAND. With $x = y = 0$, $z = 1$: $\text{NAND}(\text{NAND}(x, y), z) = \text{NAND}(1, 1) = 0$, but $(xyz)' = 1$.

02

From expressions to circuits

Drawing a circuit

Build $f = xy' + x'z$, the result of Lecture 05.



- One gate per operation: two NOTs, two ANDs, one OR.
- A dot marks where one wire branches into two; crossing wires without a dot are not connected.
- A sum of products always gives this shape: NOTs, a column of ANDs, and one OR.

Building and testing in Logisim-evolution

1. Place an input pin for each variable and an output pin for the result.
2. Add the gates from the Gates library and connect them with wires.
3. With the poke tool, set the inputs to every combination and check the output against the truth table.

> **Remark**
>
> Testing every combination is proof by exhaustion again (Lecture 03). With 3 inputs it is 8 checks; for larger circuits, Logisim-evolution can list the truth table for you (Project → Analyze Circuit).

03

Universal gate sets

Universality

Definition (Universal set of gates)

A set of gates is *universal* if every Boolean function can be built from its gates alone.

Theorem

{NOT, AND, OR} is universal.

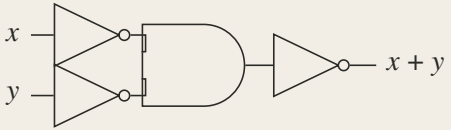
Proof. Every Boolean function has a canonical sum of products (Lecture 04), which uses only NOT, AND and OR.

NOT and AND are enough

> **Theorem**
>
> {NOT, AND} is universal.

To show a set is universal, build the gates of a set already known to be universal. Here only OR is missing:

$$
\begin{aligned}
x + y &= ((x + y)')' && \texttt{involution} \\
&= (x'y')' && \texttt{De Morgan} \\
&= \mathrm{NOT}(\mathrm{AND}(\mathrm{NOT}\, x, \mathrm{NOT}\, y))
\end{aligned}
$$



By duality, {NOT, OR} is universal too: $xy = (x' + y')'$.

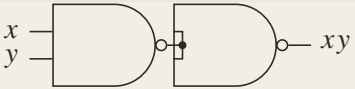
NAND alone is enough

Theorem

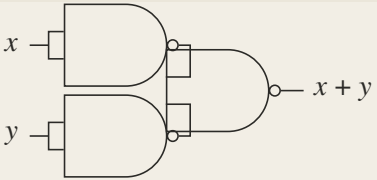
{NAND} is universal.



NOT: $\text{NAND}(x, x) = x'$



AND: a NAND, then a NAND as NOT



OR: $\text{NAND}(x', y') = x + y$

NOT and AND are enough (previous slide), so NAND is enough.

NOR alone is enough

Theorem

{NOR} is universal.

Proof.

The duals of the NAND constructions. NOT: $\mathrm{NOR}(x, x) = (x + x)' = x'$.
OR: $\mathrm{NOR}(\mathrm{NOR}(x, y), \mathrm{NOR}(x, y)) = ((x + y)')' = x + y$.
AND: $\mathrm{NOR}(x', y') = (x' + y')' = xy$.
NOT and OR are enough, so NOR is enough. □

The book builds a NAND from NOR gates instead: $xy = \mathrm{NOR}(x', y')$, then complement.

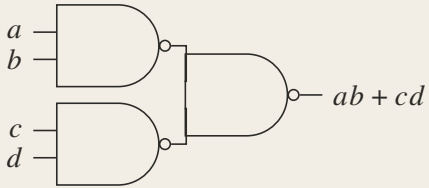
A set that is not universal

- $\{\text{AND}, \text{OR}\}$ is *not* universal: it cannot build NOT.
- Why: in a circuit of ANDs and ORs, changing an input from 0 to 1 can never change the output from 1 to 0. Each gate has this property, so the whole circuit has it.
- NOT does the opposite: as x goes from 0 to 1, x' goes from 1 to 0.

> **Remark**
>
> So the complement is what makes NAND and NOR special: each one combines an AND or an OR with a NOT.

Why NAND and NOR matter

- In CMOS, today's main chip technology, NOT takes 2 transistors, NAND and NOR take 4, and AND and OR take 6: a NAND or a NOR followed by a NOT.
- So NAND and NOR are the cheapest gates, and one gate type is enough for a whole chip.



A sum of products becomes NAND–NAND: replace every gate by a NAND.

$$((ab)' \cdot (cd)')' = ab + cd$$

De Morgan, then involution.

Try it

1. Build XOR from four NAND gates.
2. Build NOR from NAND gates only. How many do you need?
3. Is {XOR} universal?

Try it

1. Build XOR from four NAND gates.
   $n = \text{NAND}(x, y)$, $\quad x \oplus y = \text{NAND}(\text{NAND}(x, n), \text{NAND}(y, n))$. Check it on all four inputs.
2. Build NOR from NAND gates only. How many do you need?

3. Is {XOR} universal?

Try it

1. Build XOR from four NAND gates.
   $n = \text{NAND}(x, y)$, $\quad x \oplus y = \text{NAND}(\text{NAND}(x, n), \text{NAND}(y, n))$. Check it on all four inputs.
2. Build NOR from NAND gates only. How many do you need?
   Four: OR takes three (two as NOTs, one NAND), then one more as NOT.
3. Is {XOR} universal?

Try it

1. Build XOR from four NAND gates.
   $n = \text{NAND}(x, y)$, $x \oplus y = \text{NAND}(\text{NAND}(x, n), \text{NAND}(y, n))$. Check it on all four inputs.
2. Build NOR from NAND gates only. How many do you need?
   Four: OR takes three (two as NOTs, one NAND), then one more as NOT.
3. Is {XOR} universal?
   No. A circuit of XORs only ever computes the XOR of some of its inputs (or 0), and xy is not of that form.

04

Summary

Summary

- A logic gate computes a basic Boolean function in hardware: NOT, AND, OR, and the combinations NAND, NOR, XOR, XNOR.
- A bubble means complement. XOR is 1 when its inputs differ; XNOR when they are equal.
- A circuit has one gate per operation; a sum of products gives NOTs, ANDs, then an OR.
- A set of gates is universal if it can build every Boolean function: $\{\text{NOT}, \text{AND}, \text{OR}\}$, $\{\text{NOT}, \text{AND}\}$, $\{\text{NAND}\}$ and $\{\text{NOR}\}$ are; $\{\text{AND}, \text{OR}\}$ is not.

Before the next lecture

- Read Chapter 3 of the book, *Logic Gates & Circuits*.
- In Logisim-evolution, build NOT, AND and OR from NAND gates only, and test each one on every input.
- Build $f = xy' + x'z$ with NOT, AND and OR gates, and check it against its truth table.

> **Next lecture** (Unit 7)
>
> Combinational circuits: multiplexers and demultiplexers.